

IMPACT ANALYSIS

CR-019: Student Profile Editing

1. Change Summary

CR-019 introduces a student-facing profile editing facility to the Student Management System. The current StudentDashboardPage.jsx retrieves the authenticated student's profile through GET /api/students/me and displays the profile as read-only. The change will allow the authenticated student to update their email address and phone number through a controlled profile update operation.

2. Impact Assessment

Area	Impact Assessment
Frontend	Low - Moderate - Edit Profile UI, form state, validation, save/cancel controls and updated profile display will be added.
Backend	Moderate - A student-specific profile update operation is required because the existing PUT /api/students/:id route is ADMIN-only.
API	Moderate - A protected PUT /api/students/me endpoint can be added for self-update while the existing GET /api/students/me remains unchanged.
Database	Low - Existing students.email and students.phone_number fields will be updated; no schema change is expected.
Authentication	None - Existing JWT authentication remains in use.
Authorization	Moderate - The update must use the authenticated userId and prevent a student from selecting another student's record.
Performance	Low - Only one student's small profile record is updated per submission.
CRUD	Low - Existing administrator create/read/update/delete operations must remain unaffected.
Deployment	Low - Moderate - Frontend and backend code changes are required, but no database migration is expected.

3. Frontend Impact

The main affected area is `frontend/src/pages/StudentDashboardPage.jsx`.

- Add Edit Profile button.
- Add editable email and phone fields.
- Pre-populate current profile values.
- Add client-side validation and validation messages.
- Add Save/Update and Cancel controls.
- Display success/error feedback.
- Refresh the displayed profile after a successful update.
- Maintain responsive styling in `studentDashboard.css`.

The existing GET `/api/students/me` profile-loading flow and logout behaviour should remain functional.

4. Backend, API and Database Impact

The existing GET `/api/students/me` endpoint already identifies the current student using `req.user.userId`. The existing PUT `/api/students/:id` endpoint is protected by `authorizeRoles('ADMIN')`, so it should not be reused for student self-editing without weakening authorization.

- Add a protected PUT `/api/students/me` route for the authenticated student.
- Use authenticate middleware and derive the student from `req.user.userId`.
- Validate email and phone number on the server.
- Update the existing `students.email` and `students.phone_number` fields.
- Return a clear success/error response.
- Reuse `getStudentByUserId` and existing student model/database patterns where appropriate.

No database schema or migration is expected because the required fields already exist.

5. Security Impact

The change has a moderate authorization impact but does not change the existing authentication mechanism.

- Do not accept a `student_id` from the client as the authority for the update.
- Use the authenticated JWT `userId` to locate the student's own record.
- Keep the existing ADMIN-only PUT `/api/students/:id` route unchanged.
- Validate submitted values before database update.
- Do not expose or modify another student's profile.
- Keep existing JWT authentication and authorization controls in place.

6. CRUD and Functional Impact

Existing administrator CRUD operations must continue to work after the change.

- Student can view their own profile.
- Student can edit only email and phone number.
- Administrator can continue to create, view, edit and delete students.
- Student self-update must not change registration number or another student's data.
- Profile retrieval after update must return the new values.

7. Performance Impact

The expected performance impact is low. The change performs a small authenticated database update only when the student submits the form. No continuous requests are required while typing. Existing profile retrieval and administrator operations should remain unaffected.

8. Testing Impact

- Open Edit Profile from the student dashboard.
- Verify current email and phone values are displayed.
- Update with a valid email.
- Update with a valid phone number.
- Submit both valid values.
- Reject invalid email format.
- Reject invalid phone format.
- Reject empty required values.
- Verify success message.
- Verify updated values after page refresh.
- Verify database persistence.
- Verify unauthenticated update is rejected.
- Verify a student cannot update another student's record.
- Verify existing student login/profile retrieval.
- Verify existing administrator CRUD and authorization.

9. Risk Analysis

Risk	Impact	Mitigation
Wrong student profile is updated	High	Derive the student identity from authenticated req.user.userId; do not trust a client-supplied student_id.
Unauthorized profile modification	High	Protect the endpoint with authenticate and verify ownership through userId.
Invalid email/phone stored	Medium	Use frontend and backend validation before database

		update.
Existing admin update route is affected	High	Keep PUT /api/students/:id ADMIN-only and add a separate /me update route.
Profile display stops refreshing	Medium	Reload or update local student state after successful update and perform regression testing.
Responsive UI problems	Low	Test the edit form on supported desktop/mobile screen sizes.

10. Overall Impact and Recommendation

Overall Impact: Medium

CR-019 is a functional enhancement with moderate frontend/backend impact and low database impact. The required profile fields already exist, so no database schema migration is expected. The main technical and security consideration is implementing student self-update without weakening the existing ADMIN-only student update route.

The change is technically feasible and recommended for implementation, subject to formal approval and subsequent testing.

Key implementation consideration: the authenticated student's identity must be used to ensure that only their own profile is updated.